SERVICIO NACIONAL DE APRENDIZAJE – SENA

Programa de formación
Análisis y Desarrollo de Software

EVIDENCIA GA7-220501096-AA1-EV01

INFORME TÉCNICO DE PLAN DE TRABAJO PARA CONSTRUCCIÓN DE SOFTWARE

Proyecto: AeroMaint
Sistema de gestión de mantenimiento aeronáutico

Aprendiz:
Santiago Romero Sosa

2026

1. Introducción

El desarrollo de un producto de software requiere organizar las actividades técnicas y establecer mecanismos que permitan controlar los cambios realizados sobre el código fuente. En este contexto, las herramientas de versionamiento facilitan el seguimiento de las modificaciones, la recuperación de versiones anteriores, el trabajo colaborativo y la organización de las diferentes etapas de construcción del sistema.

Para el proyecto AeroMaint se propone utilizar un esquema de control de versiones basado en Git y un repositorio remoto en GitHub. Esta selección permitirá mantener una estructura organizada del código fuente, separar los cambios asociados a cada funcionalidad y conservar un historial de las modificaciones realizadas durante el desarrollo.

AeroMaint se plantea como una solución de gestión de mantenimiento aeronáutico reutilizable para una aerolínea, con capacidad para administrar múltiples flotas y diferentes tipos y modelos de aeronaves. Debido a la cantidad de módulos que contempla el sistema, resulta conveniente establecer desde el inicio un plan de versionamiento que acompañe las actividades de análisis, diseño, desarrollo, pruebas e implementación.

2. Objetivo general

Establecer un plan técnico de trabajo para la construcción del software AeroMaint, definiendo las herramientas y tecnologías de control de versiones que permitan organizar, registrar y controlar los cambios realizados durante el desarrollo del sistema.

3. Objetivos específicos

• Definir las herramientas de versionamiento que serán utilizadas durante la construcción de AeroMaint.
• Establecer una estrategia básica de organización de ramas y versiones del código fuente.
• Definir convenciones para los mensajes de los commits y el registro de cambios.
• Establecer un flujo de trabajo que facilite la integración, revisión y conservación de las diferentes versiones del proyecto.
• Relacionar el control de versiones con las etapas de análisis, diseño, desarrollo, pruebas y mantenimiento del software.

4. Descripción del proyecto AeroMaint

AeroMaint es un sistema web orientado a la gestión del mantenimiento aeronáutico. Su propósito es centralizar información relacionada con aeronaves, componentes, órdenes de mantenimiento, actividades realizadas, técnicos y usuarios, proporcionando una estructura que pueda adaptarse a diferentes flotas y modelos de aeronaves.

La solución contempla una interfaz front-end para la interacción con los usuarios y una estructura de información que posteriormente podrá conectarse con los servicios y componentes de la aplicación. Entre los módulos considerados se encuentran: Dashboard, Aeronaves, Componentes, Órdenes de mantenimiento, Mantenimiento, Técnicos y Usuarios.

La construcción progresiva de estos módulos hace necesario contar con un mecanismo que permita identificar qué cambios fueron realizados, cuándo y sobre qué funcionalidad. Git y GitHub se utilizarán como base para este control.

5. Plan general de trabajo para la construcción

Etapa	Actividades principales	Resultado esperado
Análisis	Revisión de requisitos, necesidades de usuarios y alcance de AeroMaint.	Requisitos organizados y alcance definido.
Diseño	Diseño de interfaz, estructura de componentes y organización de la solución.	Diseños y estructura técnica del sistema.
Desarrollo	Construcción progresiva de los módulos y funcionalidades.	Código fuente versionado.
Pruebas	Validación funcional, corrección de errores y revisión de cambios.	Versión validada y registro de correcciones.
Integración	Integración de funcionalidades desarrolladas y revisión de compatibilidad.	Versión integrada del sistema.
Implementación	Preparación de una versión estable para demostración o entrega.	Versión estable de AeroMaint.
Mantenimiento	Correcciones, mejoras y nuevas funcionalidades.	Nuevas versiones controladas.

6. Selección de herramientas de versionamiento

La selección se realiza considerando las necesidades del proyecto, la facilidad de integración con el entorno de desarrollo y la posibilidad de mantener un historial organizado de los cambios.

Herramienta	Función	Aplicación en AeroMaint
Git	Sistema distribuido de control de versiones.	Registrar cambios, crear ramas, comparar versiones, recuperar estados anteriores y administrar el historial.
GitHub	Plataforma para alojar repositorios y colaborar sobre el código.	Conservar el repositorio remoto de AeroMaint y facilitar colaboración y respaldo.
Visual Studio Code	Entorno de desarrollo para editar y organizar el código.	Desarrollar los archivos del proyecto y utilizar la integración de Git.
GitHub Desktop	Interfaz gráfica para operaciones frecuentes de Git.	Alternativa para visualizar cambios, preparar commits y sincronizar ramas.

7. Estrategia de ramas para AeroMaint

Se propone utilizar una estructura sencilla de ramas que permita conservar una versión estable y desarrollar funcionalidades de manera independiente.

main: rama destinada a versiones estables y entregables.
develop: rama de integración de cambios en desarrollo.
feature/*: ramas temporales para funcionalidades específicas.

Representación propuesta:

```
main
■
■■■ develop
■ ■■■ feature/aeronaves
■ ■■■ feature/componentes
■ ■■■ feature/ordenes
■ ■■■ feature/usuarios
■
■■■ versiones estables
```

8. Convenciones para los commits

Los commits deberán describir de forma breve y clara el cambio realizado. Se propone utilizar prefijos según el tipo de modificación.

Prefijo	Uso	Ejemplo
feat	Nueva funcionalidad.	feat: agregar módulo de aeronaves
fix	Corrección de un error.	fix: corregir validación de matrícula
style	Cambios visuales o de formato.	style: ajustar diseño del dashboard
docs	Cambios en documentación.	docs: actualizar documentación
refactor	Reorganización del código.	refactor: reorganizar componente de órdenes

9. Flujo de trabajo de versionamiento

Para cada funcionalidad se propone seguir un flujo ordenado: seleccionar la funcionalidad, crear o actualizar la rama, realizar los cambios, ejecutar pruebas, registrar un commit descriptivo, enviar los cambios al repositorio remoto, revisar la implementación y finalmente integrar la funcionalidad en la rama correspondiente.

Flujo resumido: **seleccionar** → **desarrollar** → **probar** → **commit** → **push** → **revisar** → **integrar** → **versionar**.

10. Organización propuesta del repositorio

El repositorio de AeroMaint deberá conservar una estructura clara para separar código fuente, documentación y archivos de configuración.

```
AeroMaint/
■■■ src/
■ ■■■ components/
■ ■■■ pages/
■ ■■■ services/
■ ■■■ assets/
■■■ public/
■■■ docs/
■■■ tests/
■■■ package.json
■■■ README.md
```

Esta organización es una propuesta inicial y podrá ajustarse a la arquitectura definitiva del proyecto.

11. Plan de versionamiento por módulos

Módulo	Rama sugerida	Cambios a controlar
Dashboard	feature/dashboard	Indicadores, tarjetas, tablas y navegación.
Aeronaves	feature/aeronaves	Registro, consulta, actualización y estados.
Componentes	feature/componentes	Trazabilidad, números de parte, series y asociación.
Órdenes	feature/ordenes	Creación, estados, prioridades y seguimiento.
Mantenimiento	feature/mantenimiento	Registro e historial de actividades.
Técnicos	feature/tecnicos	Información, licencias y especialidades.
Usuarios	feature/usuarios	Usuarios, roles y permisos.

12. Control de versiones, respaldo y trazabilidad

El uso de Git permite conservar un historial local de los cambios, mientras que GitHub proporciona un repositorio remoto para almacenamiento y colaboración. Esta combinación ayuda a reducir el riesgo de pérdida del código y facilita identificar cuándo se introdujo una modificación.

En AeroMaint, cada cambio significativo deberá quedar asociado a un commit descriptivo y, cuando corresponda, a una rama relacionada con una funcionalidad. El historial podrá utilizarse como referencia para revisar la evolución del software y apoyar las actividades de mantenimiento.

13. Integración con el proceso de desarrollo

El versionamiento constituye un elemento transversal del proceso de construcción. Durante el análisis se pueden versionar documentos y especificaciones; durante el diseño, archivos relacionados con interfaces y arquitectura; durante el desarrollo, el código fuente; y durante las pruebas y mantenimiento, las correcciones y nuevas versiones.

14. Conclusiones

El control de versiones es un componente importante para organizar la construcción de AeroMaint, debido a que permite administrar los cambios y conservar diferentes estados del proyecto.

Para el proyecto se propone utilizar Git como sistema de control de versiones y GitHub como repositorio remoto, complementados con Visual Studio Code y, de manera opcional, GitHub Desktop. Esta combinación proporciona herramientas para registrar modificaciones, trabajar mediante ramas, revisar cambios y mantener versiones estables.

La estrategia propuesta puede aplicarse progresivamente a los diferentes módulos de AeroMaint, manteniendo una separación entre funcionalidades y facilitando su posterior integración. Así, el plan de versionamiento queda vinculado con las etapas de análisis, diseño, desarrollo, pruebas, implementación y mantenimiento.

15. Referencias

Git. (s. f.). *Git Documentation*. Documentación oficial de Git.

GitHub. (s. f.). *GitHub Documentation*. Documentación oficial de GitHub.

Microsoft. (s. f.). *Visual Studio Code Documentation*. Documentación oficial de Visual Studio Code.